



Faculty of Science and Technology

BSc (Hons) Games Software Engineering

May 2025

Improving a Deep-Q Network agent to complete the Cart Pole program

by

Daniel Ellett

DISSERTATION DECLARATION

This Dissertation/Project Report is submitted in partial fulfilment of the requirements for an honours degree at Bournemouth University. I declare that this Dissertation/Project Report is my own work and that it does not contravene any academic offence as specified in the University's regulations.

Retention

I agree that, should the University wish to retain it for reference purposes, a copy of my Dissertation/Project Report may be held by Bournemouth University normally for a period of 3 academic years. I understand that my Dissertation/Project Report may be destroyed once the retention period has expired. I am also aware that the University does not guarantee to retain this Dissertation/Project Report for any length of time (if at all) and that I have been advised to retain a copy for my future reference.

Confidentiality

I confirm that this Dissertation/Project Report does not contain information of a commercial or confidential nature or include personal information other than that which would normally be in the public domain unless the relevant permissions have been obtained. In particular, any information which identifies a particular individual's religious or political beliefs, information relating to their health, ethnicity, criminal history or personal life has been anonymised unless permission for its publication has been granted from the person to whom it relates.

Copyright

The copyright for this dissertation remains with me.

Requests for Information

I agree that this Dissertation/Project Report may be made available as the result of a request for information under the Freedom of Information Act.

Signed:

A handwritten signature in black ink, appearing to read 'Daniel Ellett', is written over a light gray rectangular background.

Name: Daniel Ellett

Date: 17/05/2025

Programme: BSc GSE

Permission for use of produced work

We ask you to sign this form because Bournemouth University Higher Education Corporation (BU) may wish to use your Individual Project material (in its original or an amended form) for marketing purposes in one or more of the following ways:

- prospectuses and other BU promotional materials, including promotional videos
- as part of an advert or advertisement feature (which includes possible use on outdoor media such as buses and billboards)
- on the BU websites or intranet
- at open days
- in social media

If we used it, you would receive exposure for your work. You do not have to agree, and not doing so will have no effect on marking your Individual Project material. For ease this is simply called Material below.

If you are kind enough to agree to BU having rights of use and editing, this will not affect your ownership rights in the Material or your right to use it for your own professional or personal reasons. Unless it is impractical to do so – for example a montage on social media where space limits may apply, BU will ensure that it credits you for BU's use of the Material.

By signing this form:

- I grant a non-exclusive, world-wide, royalty-free licence to BU to use and sub-licence the Material, for marketing purposes; and I waive any non-property rights (including moral rights) now or in the future existing. This licence includes the right to edit and create derivative works from the Material, for example, a multi-student showreel.
- I agree this licence shall subsist permanently unless I seek to end it in line with the procedure below; but that where BU has already used, or is committed to using, the Material for a particular marketing campaign - for example, in a prospectus already printed, any ending of the licence shall relate only to new uses of the Material.
- I understand BU will not ask me for any further approval or permission for specific uses of the Material for marketing purposes; but that I can withdraw my permission at any time, with the consequences for continued use set out above.

If you wish to withdraw your permission for use of your information, please contact the Programme Support Officer by e-mail scitechcreativepso@bournemouth.ac.uk. Otherwise, we will keep your Material and associated personal information for as long as we have any plan to, or actively, use the Material for marketing purposes.

The legal basis for BU processing your personal information is consent, that is by signing this form you are giving permission to use your personal information provided in, and in relation to, the Material for the purposes set out above.

In processing your personal data, BU complies with the Data Protection Act 2018 and General Data Protection Regulation (Regulation (EU) 2016/679). You can find further information about BU's data protection and privacy approach here: <https://www.bournemouth.ac.uk/about/governance/access-information/data-protection-privacy>.

Further information about your rights under the data protection legislation can be found from our Data Protection Officer on dpo@bournemouth.ac.uk.

I give permission for BU to use my Material on the basis set out above:

Signed:

A handwritten signature in black ink, appearing to read 'D. Ellett', is displayed on a light gray rectangular background.

Name: Daniel Ellett

Date: 17/05/2025

Acknowledgements

I would like to thank my lecturers, in particular Simant Prakoonwit, Fred Charles and Michael Smith, who gave me guidance on my dissertation and stopped me from giving up over the course of my thesis. I would also like to thank my classmates and friends for the past three years; I would have not been able to go through the course without sharing my struggles. Particularly, I would like to thank Fin Hall, Nathan Bailey and James Comer for being not only friends but sharing all the experiences with me along the way. Finally, I would like to thank my partner Trish for the endless nights spent stressing over nothing with me.

Abstract

This project investigates the performance of three different reinforcement learning architectures – Deep Q-Network (DQN), Double DQN (DDQN) and Dueling DQN (D3QN) – in solving the CartPole-v1 benchmark from OpenAI’s Gymnasium. While DQN laid the foundation for value-based deep reinforcement learning, it has known limitations such as overestimation bias and inefficiencies in value estimation, which became clear after exploring DDQN and D3QN. The main goal of this project is to compare these architectures in training stability, learning speed and final policy performance, using consistent hyperparameter testing and controlled randomness for reproducibility.

The agents were implemented using PyTorch and trained under identical conditions across five random seeds. Each model used experience replay, target networks and various loss functions including Mean Squared Error (MSE) and Huber loss. Hyperparameters like learning rate, gamma, epsilon decay and buffer size were systematically evaluated and fine-tuned.

The results showed that DDQN reached the threshold faster but was not overly successful in its average reward, while DQN often achieved the highest average reward across seeds when using the optimal hyperparameters. D3QN showed some potential improvements but was sensitive with hyperparameter choices. Analysis showed that while DDQN in theory should have performed better, simpler environments such as Cart Pole may not be enough to demonstrate these. Loss function experiments showed that MSE and Huber loss were the most effective.

The project contributes to the understanding of architectural changes and how they affect agent performance in reinforcement learning environments. Limitations due to resource constraints and environmental complexity are acknowledged while future work in this field should be targeted towards more complex tasks like Atari benchmarks. Overall, this study offers valuable insights into the trade-offs between DQN and its variants in the field.

Table of Contents

Acknowledgements	- 5 -
Abstract	- 6 -
Table of Contents	- 7 -
Introduction.....	- 8 -
Background and related work	- 8 -
Introduction to reinforcement learning.....	- 9 -
Q-Learning and value-based methods	- 9 -
Deep Q-Networks (DQNs)	- 9 -
Double Deep Q-Networks (DDQNs)	- 10 -
Dueling Deep Q-Networks (D3QNs)	- 10 -
Cart Pole as a reinforcement learning benchmark.....	- 10 -
Key Supporting Components.....	- 11 -
Design	- 11 -
Overview and Rationale of Reinforcement Learning	- 11 -
Detailed Description of DQN Architectures	- 12 -
Cart Pole Environment	- 13 -
Hyperparameter Choices and Rationale	- 14 -
Loss Functions	- 14 -
Target Network Strategy	- 15 -
Exploration-Exploitation Strategy	- 15 -
Seed Choice and Reproducibility.....	- 16 -
Implementation	- 16 -
Software & Libraries.....	- 16 -
Code structure.....	- 16 -
Model and network architecture details.....	- 17 -
Training process	- 17 -
Practical challenges and debugging	- 18 -
Evaluation and logging	- 18 -
Experiment management.....	- 18 -
Evaluation	- 19 -
Loss Function Comparison	- 19 -
Training Performance.....	- 21 -
Statistical Summary.....	- 24 -
Strengths and Weaknesses	- 24 -
Conclusion	- 25 -
References	- 26 -
Appendices	- 26 -

Introduction

In the past few years, artificial intelligence (AI) has come into everyone's day-to-day life. A big foundational stone of AI is reinforcement learning, where its potential to allow agents to make their own decisions in environments given to them. With multiple advancements in this field, many implementations set out to solve problems with stability, learning rates and reproducibility, especially when used in continually updating environments. This project was motivated by the interest in how different architectural upgrades to a vanilla DQN network can address these problems.

The aim of this research is to evaluate and then compare three closely related algorithms – Deep Q-Network (DQN), Double DQN (DDQN) and Dueling DQN (D3QN) – in their ability to solve the Cart Pole benchmark, a task from OpenAI's Gymnasium testing environments. While each algorithm improves upon the foundation of Q-Learning, each iteration introduces unique mechanisms designed to affect performance in areas like learning accuracy and training stability.

This project aims to research how these mechanisms affect performance in a simple environment like Cart Pole, by implementing a base DQN and assessing its performance under different hyperparameters and advancements such as Double DQN and Dueling DQN. The focus of the project is its improvements in training consistency, learning speed and maximum reward performance, evaluated across multiple random seeds to ensure that its reproducible and is watching for random spikes. Not just adding on to DQN, it also explores the effect of different loss functions and buffer sizes on agent performance.

Unlike high-dimensional games such as Atari games, the Cart Pole environment offers a simple space for fast experimentation and controlled analysis. This in turn allows a better comparison of the network changes, instead of comparing how the complex environment changes the expected rewards. By doing this, the research can show how enhancements like how Double DQN value evaluation or advantage-based architectures like Dueling DQN translates into actual performance gains.

This project also allows for comprehensive reproducibility of the code. All models are trained under various identical conditions with controlled but random seeds and performance is tracked using the same performance metrics. The code is modular which supports easy extension and that any improvements can be tracked to exactly what caused such noticeable improvements.

The contributions made in this work is shown by the experiment's results, in that I show a nearly 300 average reward improvement over the base code DQN (SeeknnDestroy, 2023) in some configurations. It also contributes to the benchmark by showing my own insights into how DQN, Double DQN and Dueling DQN perform side-by-side.

Background and related work

Introduction to reinforcement learning

Reinforcement learning is a part of machine learning that deals with how agents take actions in an environment to maximise their overall reward over time. Where supervised learning is based on models learning from labelled input and output pairs, reinforcement learning is based on trial-and-error interaction with a dynamic system, receiving rewards that guide behavioural adaptation (Sutton & Barto, 2015). This allows for decision-making that affects the future, where each action not only affects the immediate outcomes but also the agent's future state and long-term performance.

Reinforcement learning problems are commonly known as a Markov Decision Processes, defined by a set of states, actions, transition dynamics, a reward function and a discount factor (Sutton & Barto, 2015). The end goal of the project is to learn a policy that maps states to actions so that the network gets the maximum expected return. Most importantly is the concept of value functions, which estimate the utility of states or state-action pairs under a particular policy. These foundational concepts form the basis for various algorithms, including Q-Learning and Deep Q-Networks, which are central to this project's focus on solving the Cart Pole benchmark.

Q-Learning and value-based methods

Value-based methods in RL revolve around estimating the expected future values for rewards that agents can obtain from a state or state-action pair. The commonly used method of value based learning is Q-Learning, which learns an action-value function that estimates the expected cumulative reward of taking action A in state S and then follows the optimal policy thereafter (Watkins & Dayan, 1992).

Q-Learning has been important in RL due to its simplicity and convergence guarantees under certain conditions. However, its original implementation uses a lookup table to store Q-values, which is not usable in environments that have large or continuous state spaces. In those cases, generalisation with function approximation is necessary.

Deep Q-Networks (DQNs)

RL moved towards high-dimensional and continuous environments, and this led researchers to find an innovative approach when Q-learning became impractical. Deep Q-Networks were introduced by Mnih in 2015 which integrated Q-learning with deep neural networks to approximate the action-value function (Mnih, et al., 2015). This allowed agents to generalise across state spaces by learning a function that maps states to Q-values for each possible action.

With DQN came two field-changing inventions to help stabilise learning.

Experience replay introduced transitions that are experienced by the agent and are then stored in a replay buffer which was sampled randomly during training. This breaks the correlation between data that comes time-step after time-step and smooths the learning curve.

Target networks were also introduced, which was a separate neural network that was not updated as regularly as the main network. It is used to compute target Q-values during training, which reduces the risk of the network losing track of its learning when updating both the prediction and target with the same parameters.

The combination of these new innovations helped address challenges in training deep RL agents and enabled new breakthroughs in performance on complex tasks, such as achieving human level performance in multiple Atari 2600 games using only raw pixel input (Mnih, et al., 2013).

Although the Cart Pole benchmark is much simpler than Atari games, its continuous state space still works with DQN's function approximation. In environments such as this, DQNs enable agents to learn robust policies efficiently without needing lower-level representations of the environment. This project uses DQN as its baseline for improvement.

Double Deep Q-Networks (DDQNs)

While DQN's introduction came with a lot of positives for the RL field, it also came with some limitations, the most prominent of these being its overestimation bias. This bias occurs when the same Q-network is used to select and evaluate actions, which can inflate value estimates and destabilise learning, especially in sparse-reward environments.

In 2016, van Hasselt proposed the Double Deep Q-Network, which decouples the action selection and evaluation processes in the learning update. In Double DQN, the online/main network selects the action that maximises the Q-value while the target network evaluates the Q-value of that action (van Hasselt et al., 2016).

This change also significantly reduces overoptimistic estimates and improves both the stability of the learning as well as its performance.

Evaluations have shown that DDQN outperforms DQN in many environments, particularly those with deceptive rewards. In this project, the Cart Pole benchmark is used to evaluate the DDQN's performance compared to the DQN. Although Cart Pole has a relatively simple reward structure, DDQN still provides more consistent training behaviour under certain hyperparameter settings.

Dueling Deep Q-Networks (D3QNs)

With both architectures having similar benefits, they share similar downsides as well. The main downside to both DQN and DDQN is their inefficiency in learning values in states where actions have little-to-no effect. To overcome this, Wang in 2016 introduced the Dueling Deep Q-Network (D3QN) which modifies the architecture of the Q-Network into two separate streams where they estimate a component each (Wang, et al., 2016).

One calculates the state-value function, representing the inherent value of being in a state while one calculates the advantage function, representing the added value of taking a specific action in that state (Wang, et al., 2016). These two streams are then combined into a single Q-value output using the following formula.

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$$

Equation for Dueling DQN single Q-value output

This allows the agent to better generalise in states where one action is not obviously the better choice than the others, which is common in benchmarks such as Cart Pole. By making state value and action advantage equations two different streams, D3QN focuses learning on which states matter, especially when action differences are minimal.

Cart Pole as a reinforcement learning benchmark

The Cart Pole environment is a widely used benchmark in reinforcement learning research. It is a "classic control environment", which is standardised in OpenAI's Gym framework (OpenAI, 2025). The goal of this benchmark is to train an agent to balance a pole hinged to a cart by applying forces

to move the cart left or right. The task terminates when the pole angle exceeds 12+- degrees or the cart moves out of bounds.

Cart Pole gives a continuous state space with four variables: cart position, velocity and pole angle, angular velocity. Despite how simple this benchmark's space is, the continuous nature of the observations makes it unusable for Q-table learning.

This means that Cart Pole is a lightweight benchmark for assessing the capabilities of value-based RL algorithms with function approximation. Its reward signal and clear success criteria make it easy for evaluation and for comparing architectures such as DQN, Double DQN and Dueling DQN.

Key Supporting Components

The stability of learning and performance of DQN agents, especially in environments like Cart Pole, depend not only on the architecture but also on the supporting components used during training. The three components that are especially crucial are the experience replay, target networks and loss functions.

Experience replay stores the agent's past interactions in a buffer and samples from it randomly. This breaks up the continuous data going into the network, improves data efficiency and reduces the huge changes in gradient updates (Mnih, et al, 2015). Without it, agents may overfit to recent transitions or fail to generalise effectively.

Target networks, which were introduced with DQN and then expanded upon in DDQN (van Hasselt et al., 2016), stabilise training by using a separate neural network to calculate target Q-values. The weights of the target network are either updated periodically, which is a hard update, or incrementally using a factor (τ) which are soft updates, which then reduces feedback loops and training spikes.

Loss functions are used to minimise the difference between predicted and target Q-values. Mean Squared Error is the standard, but Huber Loss is sometimes preferred for its robustness to outliers and ability to dampen the effect of large errors. These choices influence the speed at which the networks come together and final performance across different DQN variants.

Design

Overview and Rationale of Reinforcement Learning

Reinforcement learning is a field of machine learning in which agents learn to make decisions based on their interaction with an environment, which is Gym in this project. This differs from supervised learning, where models learn from labelled inputs and outputs, because reinforcement learning involves learning through trial and error, updating itself by using a reward (Sutton & Barto, 2015). The agent observes the current state of an environment and selects an action and then receives feedback in the form of a reward and then goes to the next state. Over time the agent will learn an optimal policy to gain these rewards and a path that maximises those rewards (Sutton & Barto, 2015).

A big part of algorithms in reinforcement learning is Q-learning, which is a value-based approach where the agent estimates the quality of a specific action in a state (Watkins & Dayan, 1992). The Q-values are updated using the Bellman equation, which uses immediate rewards and maximum expected future rewards from the agent's next state. This allows the

agent to gradually learn which actions are more beneficial over the long term. Q-learning does not need to know about the environment, and this makes it adaptable to complex systems that are unknown to it (Watkins & Dayan, 1992).

Q-Learning usually relies on a Q-Table which stores the estimated values for every possible pair of action-states. This is only effective for small environments, as when it is large or continuous it becomes impractical (Mnih, et al., 2015). In the cart pole problem, four different variables would lead to a massive number of possible states. This results in large memory requirements.

To address this, deep learning techniques – specifically neural networks – are employed to estimate the functions. Instead of storing Q-values in a table, a neural network is trained to estimate the Q-function – it takes each state as an input and outputs Q-values for each possible action. This architecture, known as a Deep Q-Network, enables the agent to generalise previously unseen states based on the previously seen states (Mnih, et al., 2015). This allows the agent to learn in continuous and large environments. Neural networks also allow the network to be scalable and flexible, which in turn allows reinforcement algorithms to be used for different tasks, such as playing games.

Detailed Description of DQN Architectures

This project aims to improve upon existing code sourced from a GitHub repository (SeeknnDestroy, 2023), extending it across three different branches: Deep Q-Network (DQN), Double DQN (DDQN) and Duelling DQN (D3QN). Each of these reinforcement learning methods represent small but noticeable improvements in the efficiency, stability and the accuracy of the Q-learning agents. The primary goal of this project is to solve the Cart Pole (v1) problem, which is a well-known benchmark for OpenAI's Gym library. In this benchmark, the agent learns to balance a pole vertically on the cart that moves horizontally along the x-axis. The agent gets observations from the environment such as the cart's position, the cart's velocity, the pole's angle and the poles angular velocity. Based on these observations, the agent must decide whether to go left or right at each timestep, making the pole balance for as long as possible. By comparing these different networks, the project's goal is to analyse how different algorithmic improvements change the learning performance in a benchmark.

DQN serves as the main foundation for architectures in this project, using a deep neural network to approximate the Q-function. It is an effective baseline for comparison with more advanced variants due to its generalisation abilities across state spaces, especially in the Cart Pole environment.

In DQN, the main objective is to minimise the difference between the predicted Q-value for a pair and the target Q-value, which is worked out using the Bellman equation. This is achieved by using a loss function, such as Mean Squared Error or Huber loss, to estimate the difference between the current Q-value and the target value. In this DQN, it also uses an experience replay, where past transitions are stored in a replay buffer and sampled randomly to train the network. This helps reduce mistakes between observations and makes the learning more efficient.

Another key part of the network is the target network, which is a secondary neural network that is only updated periodically with the weights of primary Q-network. This helps to stabilise the target and prevents derailing feedback loops during training.

Double DQN extends the standard DQN architecture by solving its main problem – overestimation bias. This bias happens when the same DQN network is used to select and evaluate the best action, which leads to inflated Q-values and unstable learning. The Double DQN algorithm introduces two separate networks to split the decision-making process. The online network is used to select the action with the highest predicted Q-value while the target network evaluates the value of that action. This usually results in higher accuracy and a less biased estimate of the target Q-value during training (van Hasselt et al., 2016). This leads to more stable learning and improved performance, especially in environments with little to no rewards and high variance in actions. In the project, the double DQN demonstrated improved learning stability and consistency aligning with the project’s objectives.

Duelling DQN further enhances performance to the original DQN by changing the structure of the network to better predict Q-values when the actions taken have minimal influence on the overall outcome. Instead of outputting values directly per action, it splits the network into two separate streams – one that estimates the value function and the other estimates the advantage function which represents the added value of taking each action in a state, compared to other actions. These two streams are then combined to produce the final Q-value for each action.

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$$

Equation for producing final Q-value.

In a normal DQN network, summing the value and advantage functions is not possible. As the value and action variables are unidentifiable (Wang, et al., 2016). Duelling DQN ensures that the output is identifiable, and this avoids redundancy in estimation. In this project, Duelling DQN helps the agent learn more efficiently when the minor changes in actions result in similar outcomes.

Cart Pole Environment

The Cart Pole benchmark environment is a control problem and is widely used in reinforcement learning experiments and research. The project uses the OpenAI Gymnasium library which simulates a system where the objective is to balance a pole on a moving cart and not let it fall. The agent uses this environment by moving the cart to the left or to the right, which are the main learning outcomes. The environment simulates physical dynamics such as gravity and momentum, which allows it to benchmark the agent’s decision making by creating a continuous and real-time decision-making process.

The benchmark presents an observation space consisting of the cart position, cart velocity, pole angle and the poles velocity. All of these values are bound by the simulation parameters. The action space only has two possible actions, pushing the cart left or right,

which makes it a discrete space. Per time step, the agent receives a reward of 1 if the pole is upright, which encourages the agent to have longer and longer episodes. These episodes end when the pole falls more than 12 degrees from upright, the cart moves out of scope or if the episode limit is reached. The episode also ends when the problem is solved which, by OpenAI's guidelines (OpenAI, 2025), is when the average reward of the agent is 475 or above for a period of 100 consecutive episodes.

Hyperparameter Choices and Rationale

Hyperparameters play a significant role in working out the learning efficiency, stability and success of the agent. This project needed to select appropriate hyperparameters to optimise agent performance across learning rate, gamma, epsilon decay rate and replay buffer size.

The learning rate controls how much the network's weights are adjusted each time the agent updates. A lower learning rate could lead to the network taking longer to combine while a higher rate could lead to unstable learning. This project extensively tested multiple learning rates – $1e-4$, $5e-4$, $1e-3$ and $3e-3$ – to evaluate its effect on the networks performance. The results of this showed that lower rates led to more stable training across DQN, DDQN and D3QN.

The discount factor (gamma) controls how much the agent values future rewards compared to ones that happen to it immediately. Values that are closer to 1 are for long-term gains and, for Cart Pole benchmarking, this is important because it must balance the pole for a prolonged period of time. In this project, gamma values were tested from 0.90 to 0.99 and it found that the lower gamma values resulted in shorter episodes and poorer long-term performance.

The epsilon decay rate (EDR) is responsible for the greedy policy of the network. The epsilon greedy policy is a decision-making strategy used to balance both exploration and exploitation. This policy will choose the best-known action most of the time, which is exploitation, while occasionally selecting a random action, which is exploration, to discover better options. The agent starts by taking random actions when the EDR is closer to 1 and gradually shifts towards exploitation as the epsilon decays over the agent's episodes. In this project, there were multiple tested decay rates – 0.990, 0.993, 0.996 and 0.999 – and this showed that a faster decay led to the network using a worse policy but was faster while a slower decay allowed the agent to explore and undertake more robust policy learning.

The replay buffer size holds how many past experiences the agent keeps for its training. Large buffers offer a wider range of training and reduce correlations between data but require more computer memory and can invalidate recent experiences. The smaller buffer however offers faster updates and the data that is newer however this risks overfitting the agent to recent network patterns. In this project, each buffer size was tested across 5 random seeds which measures consistency in rewards and learning speed across the network.

Loss Functions

The choice of loss function in reinforcement learning projects determines how effectively

the model learns from its prediction errors. The loss function works out the difference between the predicted Q-values and the target Q-values, which are worked out by using the Bellman equation.

$$V(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s') \right]$$

The Bellman equation, where Vs is the value of the state, $R(s, a)$ is the reward, γ is the discount factor, $P(s, a)$ is the probability between the states using the action, $\max(a)$ is the optimal action.

In DQN, Mean Square Error is the most-used loss function as it is strict with large deviations, encouraging the model to learn stable and accurate value predictions. However, Mean Squared Error can be sensitive to outlying data and lead to unstable updates when Q-value targets change massively during training.

In my project, multiple different loss functions were tested, including Huber loss, L1 loss, cross-entropy, KL loss and hinge embedding loss. Huber loss is the common replacement for Mean Square Error as it combines parts of L1 and Mean Square Error to lessen the impact of large errors while still valuing small errors the same (van Hasselt et al., 2016). The project also explored KL and cross-entropy, which is used for classification tasks, just to observe the training the network would have with these to exhaust all the options.

The project results showed that MSE and Huber loss led to the highest average rewards in the quickest time across 5 random seeds, while cross-entropy and KL did not peak an average reward of 25 due to their suitability for this project.

Target Network Strategy

The use of a target network is to improve the stability of learning in a DQN. In a DQN network, the target Q-value used in the Bellman equation is taken from the same network that's being updated. This leads to feedback loops and huge jumps in training, especially when using non-linear function approximators such as neural networks. To fix this, DQN introduces a second network called a target network to compute the target Q-value during learning updates while the main network is used to select actions and learn.

Instead of updating the target network per time step, its weights are updated towards the main networks weights at scheduled regular intervals using a small factor, usually around 0.001, written as tau or τ . This update rule, called a soft update, is written as:

$$\theta_{\text{target}} = \tau * \theta_{\text{current}} + (1 - \tau) * \theta_{\text{target}}$$

The soft update equation where ϑ represents the network parameters.

This process is slow and helps reduce variance in the results and improves the stability of the two streams converging.

This results in the effectiveness of target networks being high and that it is now considered a standard component in all modern DQN variants (Mnih, et al, 2015). In the project, it is used in DQN, Double DQN and Duelling DQN and it is a vital part of stable and consistent learning.

Exploration-Exploitation Strategy

In all projects, agents must balance both exploration, which is selecting a random action, and exploitation, picking the best-known action, to achieve maximum reward outcome per

time step. Especially in initial stages of the network, when not enough exploration can cause the agent to use worse policies while not enough exploitation could waste resources and time before picking the best policy. There are multiple strategies to manage this however, including Boltzmann exploration, upper confidence bound and Thompson sampling. This project used epsilon greedy policy because of its simplicity, while being highly effective. This project used an epsilon greedy policy for DQN, DDQN and D3QN which kept the data easily comparable. This allowed the agent to explore the environment before slowly focusing on exploiting the policy it chose. In the results of the project, the slower decay rates were found to be more stable and support better learning in the low dimension and structured environment that it was trained in. This shows its suitability for environments with well-defined rewards and small managed state-action spaces. (Sutton & Barto, 2015)

Seed Choice and Reproducibility

In this project, seeding is done to generalise results from testing. This project evaluates each network across 5 seeds which reduces the influence of outliers or anomalous training runs when averaging across seeds. By setting and recording seed values, the work in this project can be replicated under the same conditions to compare performance fairly and test different methods in the same environment. Multiple seed evaluation can also show if an algorithm is reliable in its performance or if it is a lucky outlier.

Implementation

Software & Libraries

This project was developed in Python, which is the industry standard programming language in machine learning due to how simple it is, while remaining both flexible and extensive with its range of libraries. The deep learning framework used was PyTorch, which was chosen for its dynamic computation graph, its flexibility and its high speeds for implementing deep neural networks which then allowed for efficient construction and training of the neural networks used in each agent. To create the learning environment in the project, Gymnasium from OpenAI was used because it offered a standardised API and a large range of benchmark environments, from which the project used Cart Pole (v1), and this allowed Gymnasium to simplify the interaction between the agent and the environment.

Other libraries used in this project included NumPy, which allowed numerical operations; Matplotlib, which allowed the project to produce charts and graphs; and Imageio, which was used to create videos and gifs for demonstration.

Code structure

The project was modularised to keep the code both readable and scalable. In the project, it was also easy to modify the code to experiment, which helped to build the DDQN and D3QN. It consists of three main classes: DQNAgent, QNetwork and ReplayBuffer.

The DQNAgent class contains the learning logic of each architecture (DQN, Double DQN and Dueling DQN). It includes methods for acting in the environment, storing the experiences the agent has, sampling from the memory and updating network parameters. The agent maintains an online Q-network and target Q-network, which allows for soft updates and Double Q-learning.

The QNetwork is a fully connected feedforward neural network. For DQN and Double DQN, the network is made up of two hidden layers with ReLU activations followed by an output layer with the

same number as possible actions in the space. For Dueling DQN, the architecture splits after the second hidden layer into two separate streams – one for estimating the state-value function and the other for computing the advantage function, which then combine again to make the Q-values.

The ReplayBuffer is implemented using Python's deque data structure allowing for efficient storage of experiences. The buffer supports random sampling, which is important for breaking correlations in the training data. This contributes to improved learning stability and allows it to learn from a large pool of past experiences.

Training and evaluation were handled in separate, dedicated code cells that organised episodes, handled the epsilon greedy action selections, applied the training updates every few steps and recorded, charted and displayed performance in the form of charts and graphs with Matplotlib. This logged episode rewards and average rewards over the last 100 episodes.

Model and network architecture details

All of the agents in this project – DQN, Double DQN and Dueling DQN – are built on a core base neural network structure which was designed to approximate the Q-function. The network is implemented by PyTorch and is structured as a fully connected feedforward neural network. The input layer accepts a four-dimensional state vector from the Cart Pole environment, representing the cart's position and velocity as well as the pole's angle and angular velocity.

The hidden layers consist of two linear layers with 64 units each, using ReLU activation functions. ReLU was chosen due to its efficiency in computation and effectiveness in reducing the vanishing gradient problem, allowing for stable and efficient learning (Hu, et al., 2018). The output layer changes slightly between architectures. For DQN and Double DQN, the output layer is a single layer that produces one Q-value per possible action, which in Cart Pole is two total, to push the cart left or right.

In the Dueling DQN architecture, the network splits into two after the second hidden layer. One stream estimates the state-value function while the other estimates the advantage function. These two outputs are then combined again after.

All weights were initialised using PyTorch's default settings. This shared architecture in the networks ensured that all the comparisons made between the agents were consistent, with architectural differences only when necessary for the agent.

Training process

In this project, each time the agent would train it followed a structured and consistent routine to ensure that it could be compared to other agents. Each agent interacted with the Cart Pole environment in episodes and it selected actions based on an epsilon greedy policy, received rewards and stored transitions in a replay buffer. An episode began with the environment resetting to default and continued until the pole fell or the cart moved out of its bounds.

At each time-step, the agent observed the current state, chose an action and then recorded the experience it gained into the replay buffer. Training updates were not performed every step, instead being updated every 4 steps once there had been a minimum number of experiences collected. This delay made sure that the buffer contained a large set of transitions before beginning backpropagation.

Each time the agent updated, it sampled a minibatch of 64 random experiences from the replay buffer. They were used to compute target Q-values using the Bellman equation for DQN and, with

some changes, Double DQN.

The predicted Q-values for each sampled state-action pair were then compared to the target Q-values using the Mean Squared Error loss function before performing a gradient descent step.

Every few steps, a soft update was applied to the target network, bringing it slowly closer to the main network's weights. This improved the stability of the agent's learning by preventing the online network, which changed often, from influencing the target Q-values too strongly.

This training loop was repeated across 5 seeds and continued until the average reward over 100 episodes exceeded 475.

Practical challenges and debugging

When developing this project, several challenges came up that needed to be dealt with. One of the most common issues that came up in this project was the network's sensitivity to hyperparameter choices for training it. There were multiple tests on the benchmark that went over the original episode cap the project had pre-defined. After consideration, this hard cap on episodes was removed and training was allowed to continue until it hit the goal. These issues were identified through unusually low rewards across multiple episodes, which required further hyperparameter tuning.

Another challenge the project faced was to do with experience replay sampling. In the project, a minimum buffer size had to be implemented due to the agent trying to train on little to no data, leading to ineffective gradient updates. These issues were solved by a 1000 buffer minimum, and it displayed results immediately after implementation.

A further challenge the project faced was memory management. As the project was running all three python notebooks at once, the buffer sizes had to be changed so it did not use all the RAM on the system. This led to multiple different buffer sizes being used and tested against, each affecting how the program ran.

Evaluation and logging

An extensive evaluation and logging system was implemented to take and chart agent performance across different episodes of training. This ensured consistency and it massively helped post-training analysis. During each episode, the agent's total reward was tracked and logged, which provided a measure for how long the pole was balanced. These rewards were added to a list and averaged over the most recent 100 episodes to measure learning progress, which was important to help assess whether the agent had solved the benchmark by reaching the reward threshold of 475.

Logging was done by Matplotlib, which was used to generate visualisations of training curves, plotting per episode rewards and 100-episode averages, which allowed for visual analysis on the learning rate and any problems arisen. These graphs were essential for diagnosing underperforming networks and identifying the impact of hyperparameter changes.

To perform model comparison, trained network weights were saved allowing for reproducibility in a final evaluation cell. Final evaluation was conducted over 300 test episodes per seed, using a 0.0 epsilon-greedy policy to allow for pure exploitation, to assess the learned policy's stability and effectiveness.

Experiment management

This project's management of the experiment was vital to the fair comparison of networks. To

ensure consistency across the project, all experiments were using the same training parameters, apart from changing hyperparameters or the network itself – DQN, Double DQN and Dueling DQN – one at a time. This approach allowed problems stemming from hyperparameters or networks to be diagnosed easily.

Each training run is identified by the network used along with the seed number, while associated outputs were saved in a research document. To allow these tests to be repeated, the randomness in the environment, PyTorch and NumPy was controlled by the seed setup at the beginning of each run. This includes the use of five fixed seeds per run which provided reliable statistics from each network test.

The final results were aggregated and then analysed using the cell in the code that houses the performance metric logging, such as average reward over test episodes. These results were summarised in a table along with the graphs to allow for model comparison. This approach limited human error, ensured consistency across different evaluations and enabled reproducible conclusions about each model’s strengths and weaknesses.

Evaluation

Loss Function Comparison

In this project, the choice of loss function significantly influences the performance of the agent. Six different loss functions were tested: Mean Squared Error (MSE), Huber Loss, L1 loss, cross-entropy, Kullback-Leibler (KL) divergence and hinge embedding loss (HEL). Each model was evaluated over 5 seeds for 300 episodes on the base DQN with results summarised in a short table [See Table 1].

DQN	Loss Function	Average reward over 300 episodes
DQN	kl_div	Seed 0: Average reward over 300 test episodes: 9.28
	kl_div	Seed 1: Average reward over 300 test episodes: 9.33
	kl_div	Seed 2: Average reward over 300 test episodes: 9.29
	kl_div	Seed 3: Average reward over 300 test episodes: 9.37
	kl_div	Seed 4: Average reward over 300 test episodes: 9.31
DQN	l1_loss	Seed 0: Average reward over 300 test episodes: 18.97
	l1_loss	Seed 1: Average reward over 300 test episodes: 55.32
	l1_loss	Seed 2: Average reward over 300 test episodes: 65.28
	l1_loss	Seed 3: Average reward over 300 test episodes: 58.40
	l1_loss	Seed 4: Average reward over 300 test episodes: 9.39
DQN	cross_entropy	Seed 0: Average reward over 300 test episodes: 9.28
	cross_entropy	Seed 1: Average reward over 300 test episodes: 9.39
	cross_entropy	Seed 2: Average reward over 300 test episodes: 9.37
	cross_entropy	Seed 3: Average reward over 300 test episodes: 10.41
	cross_entropy	Seed 4: Average reward over 300 test episodes: 9.39
DQN	hinge_embedding_loss	Seed 0: Average reward over 300 test episodes: 9.28
	hinge_embedding_loss	Seed 1: Average reward over 300 test episodes: 9.33
	hinge_embedding_loss	Seed 2: Average reward over 300 test episodes: 9.37
	hinge_embedding_loss	Seed 3: Average reward over 300 test episodes: 9.37
	hinge_embedding_loss	Seed 4: Average reward over 300 test episodes: 9.31
DQN	mse_loss	Seed 0: Average reward over 300 test episodes: 449.43
	mse_loss	Seed 1: Average reward over 300 test episodes: 136.87
	mse_loss	Seed 2: Average reward over 300 test episodes: 177.88
	mse_loss	Seed 3: Average reward over 300 test episodes: 204.35

	mse_loss	Seed 4: Average reward over 300 test episodes: 187.71
DQN	huber_loss	Seed 0: Average reward over 300 test episodes: 101.70
	huber_loss	Seed 1: Average reward over 300 test episodes: 125.88
	huber_loss	Seed 2: Average reward over 300 test episodes: 135.04
	huber_loss	Seed 3: Average reward over 300 test episodes: 105.75
	huber_loss	Seed 4: Average reward over 300 test episodes: 110.33

Table 1

As expected, MSE and Huber loss had the best results out of these functions. MSE produced the highest peak average reward at 449.43 and showed that it could be used for this project in multiple seeds [See Figure 2], however it did show that it had outliers, as seen in seed 0, where it over-performed. Huber loss [See Figure 3] offered a more consistent set of rewards across all seeds, where it excels in not training outliers.

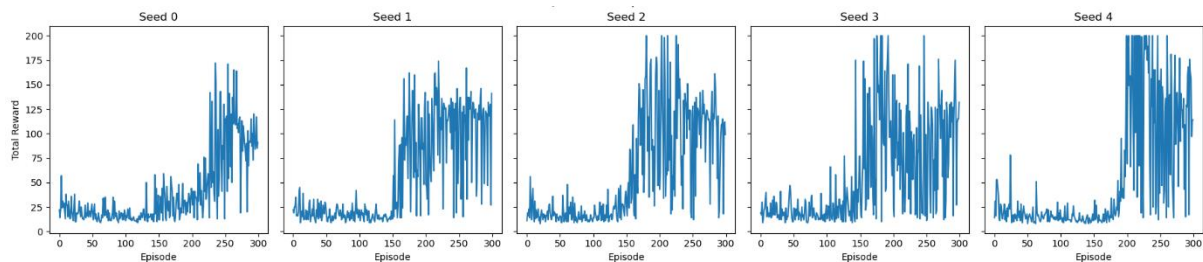


Figure 2

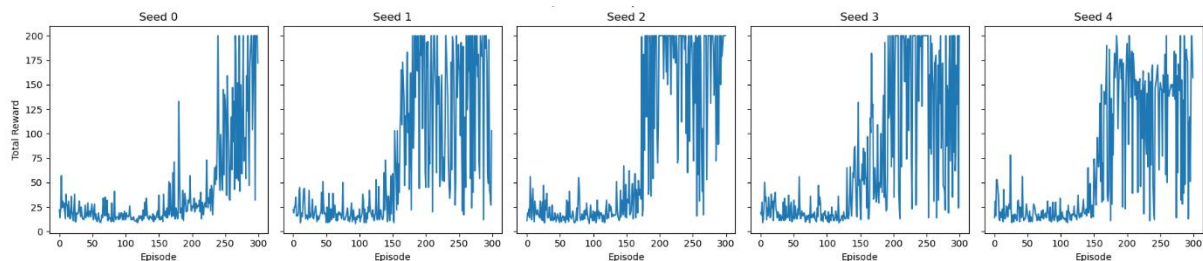


Figure 3

Classification-oriented loss functions, such as cross entropy and KL divergence, did not manage to train the program effectively. Average rewards for these functions remained around 9 to 10 over 300 episodes, with flat learning curves and erratic episode performance [See Figure 4]. This was expected because these functions are designed to optimise probability distributions rather than scalar values, such as Q-values. In Q-learning, the target is a single numeric value, not a class label or probability distribution over actions. These loss functions do not work with the regression nature of Q-value estimation, causing the network to receive distorted gradients that do not lead to good convergence rates.

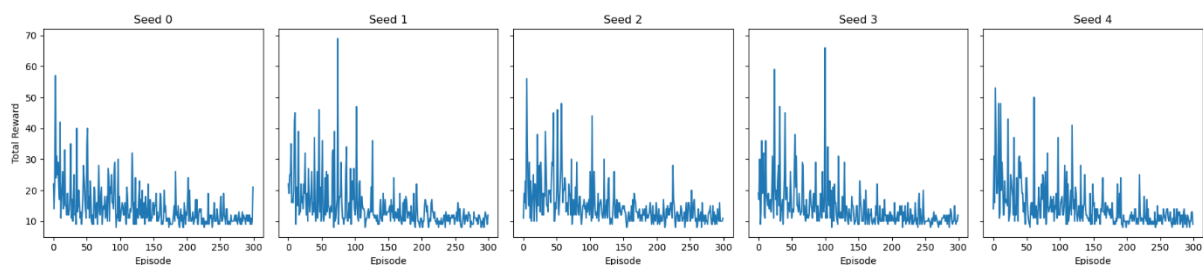


Figure 4

L1 loss showed middling results, where it occasionally exceeded 50 and then failed to achieve stable learning after that [See Figure 5]. These functions lack the power to expand on positive learning

rates and cannot fine-tune Q-values in continuous learning settings.

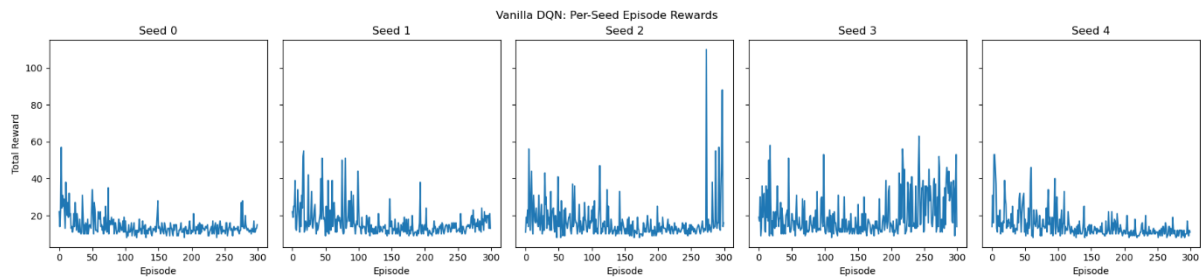


Figure 5

Overall, the results showed that regression-based loss functions such as MSE and Huber are required to successfully complete these types of benchmarks. While Huber offers a viable alternative, MSE was the strong default choice.

Training Performance

When training the networks, the performance of each agent was monitored through two performance metrics: reward per episode and average reward over the last 100 episodes. This approach allowed testing to be, for both short-term performance spikes and long-term learning trends, easily visualised. The baseline code reached an average reward around 185.08 over 100 test episodes (SeeknnDestroy, 2023). A target average reward of 250 over 100 episodes was used as the success criteria, which is lower than Open AI Gym's CartPole-v1 benchmarks, this was due to lesser training times allowing for more expansive testing.

In testing, the vanilla DQN architecture showed itself to be the most middling network, reaching the threshold in multiple runs and under various hyperparameters. Its best configuration (LR: 1e-4, Gamma: 0.99, EDR: 0.999, Buffer size: 2500 [See Figure 6]) achieved the threshold in an average of 1793 episodes across 5 seeds. Multiple other DQN configurations also showed strong learning, such as the learning rate change (LR: 5e-4, Gamma: 0.99, EDR: 0.999, Buffer Size: 2500 [See Figure 7]) and buffer size change configuration (LR: 1e-4, Gamma: 0.99, EDR: 0.999, Buffer size: 5000 [See Figure 8]).

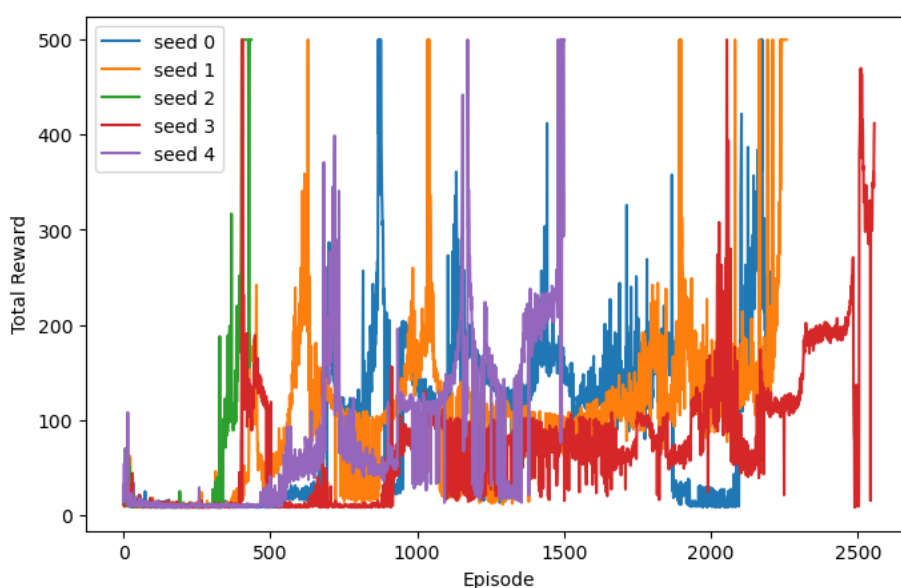


Figure 6

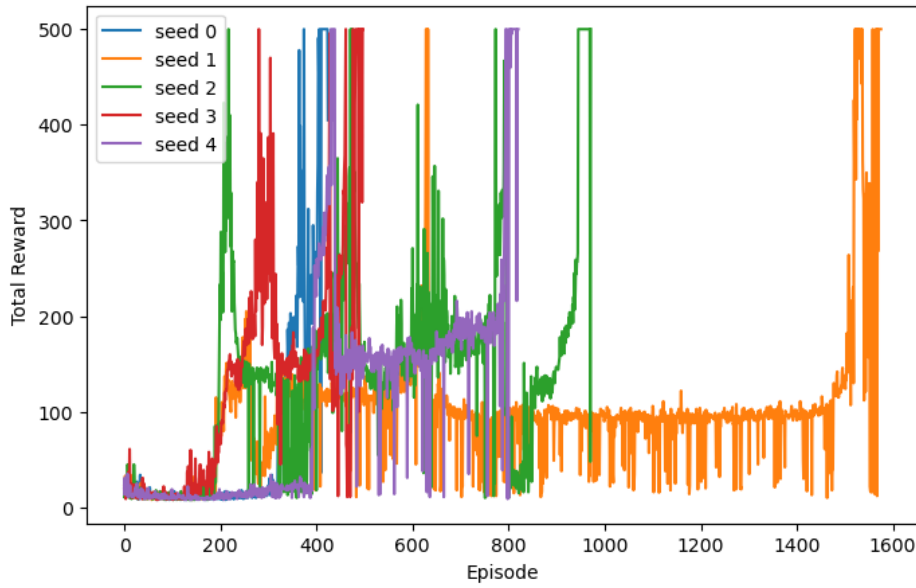


Figure 7

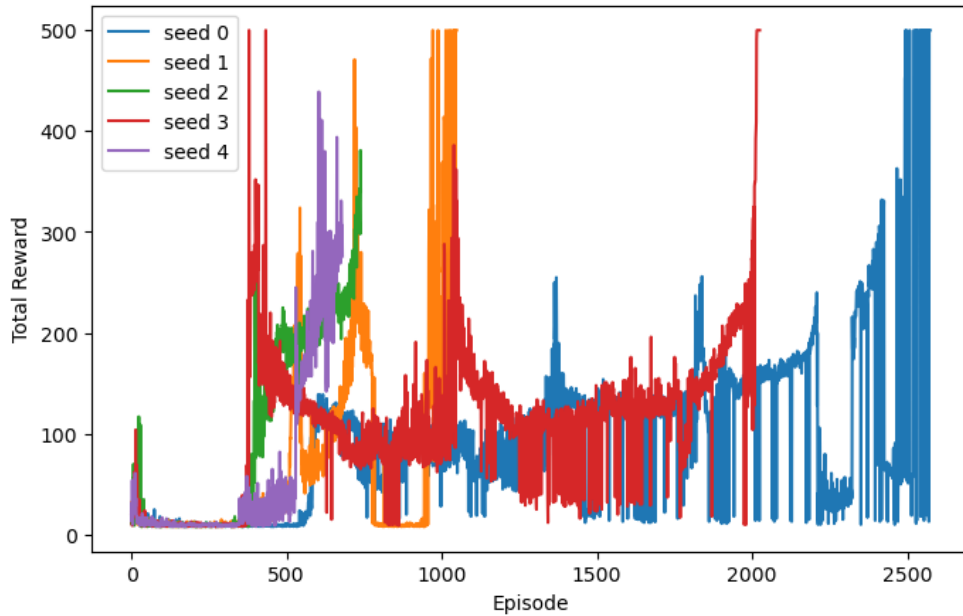


Figure 8

Compared to the DQN, Double DQN performed much better when tested, achieving lower episodes counts to reach the threshold in nearly every tested configuration [See Table 9] and this showed its advantages. It often showed up as best, showing that there are still valid use cases for it in training networks against benchmarks. Its best recorded training session reached the threshold and took 750 episodes to do so, whereas DQN was finished in 1253 and D3QN in 1690.

LR	Gamma	EDR	Buffer Size	DQN: How many episodes it took	Double DQN: How many episodes it took	Dueling DQN: How many episodes it took
1e-4	0.99	0.999	2500	1793 episodes avg	1255 episodes avg	1327 episodes avg
5e-4	0.99	0.999	2500	858 episodes avg	750 episodes avg	774 episodes avg
1e-4	0.96	0.999	2500	1253 episodes	1084 episodes	1690 episodes

				avg	avg	avg
1e-4	0.99	0.993	2500	1812 episodes avg	2143 episodes avg	1739 episodes avg
1e-4	0.99	0.999	5000	1413 episodes avg	1416 episodes avg	773 episodes avg

Table 9

Dueling DQN (D3QN) showed that it could compete with DQN and DDQN, especially in two configurations. When trained with a faster epsilon decay rate (LR: 1e-4, Gamma: 0.99, EDR: 0.993, Buffer size: 2500 [See Figure 10]), it reached the threshold inside 1739 episodes on average and when trained with a higher buffer size (LR: 1e-4, Gamma: 0.99, EDR: 0.999, Buffer size: 5000 [See Figure 11]), it reached the threshold inside 773 episodes on average. In these cases, the network showed smoother reward curves and less volatility in its rewards during mid-to-late stage phases of training.

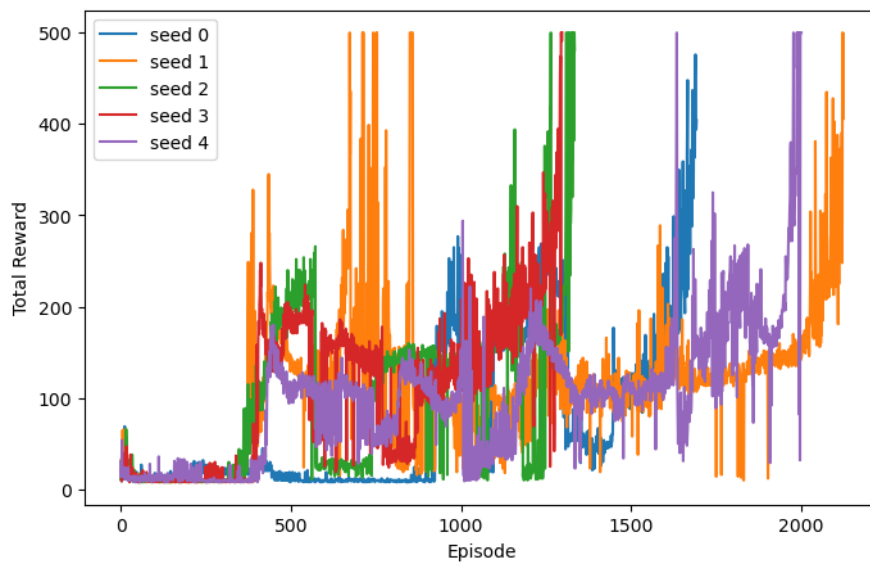


Figure 10

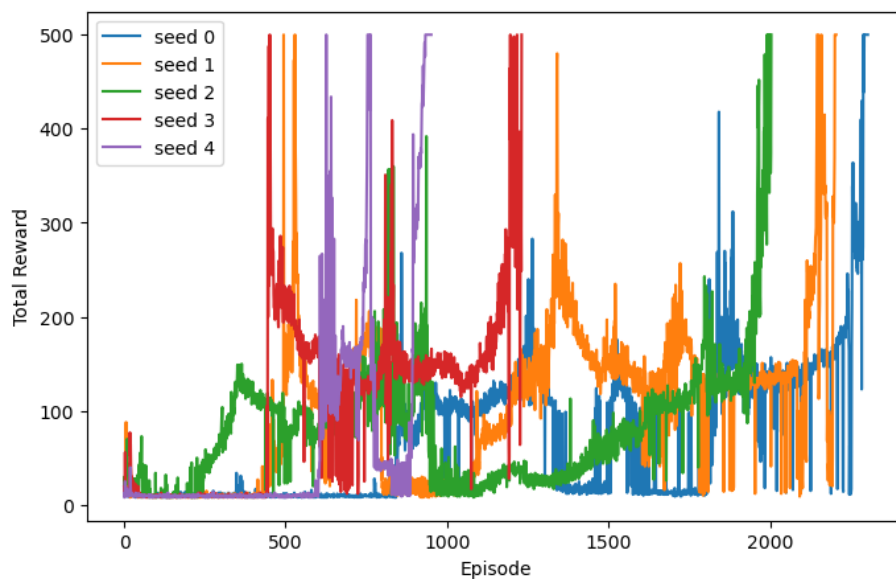


Figure 11

While all agents were able to eventually solve the task, Double DQN consistently was the best in achieving the threshold in the lowest amount of time when considering learning speed and stability, making it the most effective network across all the experiments.

Statistical Summary

To evaluate the training performance of each agent architecture, a further test was conducted over 300 episodes per seed using a pure greedy policy where $\epsilon = 0$, to find which architecture trained the best. The table [See Table 12] shows the average reward over 5 random seeds over 300 episodes at this greedy policy, with a best network column to show which network trained the best.

LR	Gamma	EDR	Buffer Size	DQN Average over 5 seeds	Double DQN Average over 5 seeds	Dueling DQN Average over 5 seeds	Best network
1e-4	0.99	0.999	2500	472.898	436.208	359.618	472.898 (DQN)
5e-4	0.99	0.999	2500	368.75	357.784	342.974	368.75 (DQN)
1e-4	0.96	0.999	2500	385.938	402.054	403.56	403.56 (Duel DQN)
1e-4	0.99	0.993	2500	363.296	439.046	467.986	467.986 (Duel DQN)
1e-4	0.99	0.999	5000	415.796	366.032	368.344	415.796 (DQN)

Table 12

The standard DQN achieved the highest average rewards across most configurations. Its top performance was with the best hyperparameters (LR: 1e-4, Gamma: 0.99, EDR: 0.999, Buffer Size: 2500) achieving an average reward of 472.898. Even with increased buffer size or a lesser learning rate, the DQN achieved the best across the networks tested with 415.796 and 368.75, respectively. This shows that despite lacking enhancements such as Double or Dueling DQN, DQN still provides a strong and stable learning environment.

Compared to DQN however, Double DQN underperformed, finishing second in 4 out of 5 tests even though in 3 out of 5 tests, it reached the threshold the quickest. Its lower reward ceiling compared to DQN and Dueling DQN shows that, although in theory it reduces overestimation bias, in practice these benefits do not translate to a higher average.

Dueling DQN performed really well in some settings, reaching 467.986 when given a faster epsilon decay rate (LR: 1e-4, Gamma: 0.99, EDR: 0.993, Buffer size: 2500) and reaching 403.56 in a lower gamma environment (LR: 1e-4, Gamma: 0.96, EDR: 0.999, Buffer size: 5000). This shows that Dueling DQN is very sensitive to parameter changes however, and when tuned properly, it can offer massive improvements on both DQN and Double DQN networks.

Overall, these results suggest that even though enhanced architectures offer some performance boosts, DQN can still be a competitive option and in most cases superior when given optimal hyperparameters. This data also shows that fine tuning hyperparameters gave performance boosts and these varied massively between configurations.

Strengths and Weaknesses

This project had multiple strengths and weakness in its implementation and its training. One strength was the use of multiple random seeds, which provided a good base for evaluating performance consistency and generalising the results. This reduced the likelihood that results were positive or negative due to randomness or a good initialisation in the environment. As well as this, training being so well documented allowed for in-depth comparison between models, helped by the criteria for completing the benchmark being straightforward so the project could define if it passed or failed.

Another strength is the testing of multiple loss functions that added more data which could be analysed. These experiments were the reason that the project could do well, as it exposed limitations of some loss functions while showing other loss functions as far superior.

Despite these strengths, several limitations also showed themselves. One noticeable limitation was the performance jumps across seeds, particularly in ones with high learning rates. Some agents showed erratic reward spikes which suggests that certain hyperparameters were causing these jumps.

Another limitation was that Double DQN did not perform how it theoretically should have, possibly due to how simple the Cart Pole environment was, where overestimation isn't as common. While it was still good to include it for comparison, it would have performed better in a higher-complexity task.

Finally, resources hindered the project. Only one computer was used, with limited specs. While five seeds were tested, this project could've benefited from more loss functions tested thoroughly or with higher testing reward threshold.

Conclusion

This project's aims were to perform tests on and evaluate the performance of three different reinforcement learning architectures – DQN, Double DQN and Dueling DQN – on the CartPole-v1 benchmark. The main objective was to take an existing code base, implement improvements in Double DQN and Dueling DQN, test their learning capabilities under different hyperparameters and then analyse their performance.

The way this was implemented was using Python, using PyTorch and OpenAI's Gymnasium environments, with each model trained across five random seeds and multiple combinations of hyperparameters. The architectures used the same feedforward neural network but used different action value calculations: DQN used a single Q-function estimator; Double DQN used a separate action selection and evaluation to reduce overestimation; and Dueling DQN used different estimators for state value and action advantage.

When testing, the results showed that the vanilla DQN consistently delivered the most stable and highest performing results, especially when using the "best" hyperparameters (LR: 1e-4, Gamma: 0.99, EDR: 0.999, Buffer Size: 2500). DQN did not always reach the threshold the fastest, but it offered the most stable, highest achieving learning across most configurations.

Double DQN, while in theory superior to DQN, underperformed in this benchmark testing environment, most likely due to the low complexity of the environment. Comparably, Dueling DQN showed high learning rates across certain hyperparameters, though it was sensitive to these hyperparameters and less consistent across the random seeds.

This project expanded my knowledge of reinforcement learning and gave me hands-on experience with using DQNs and its variants. I also gained experience in debugging performance issues, which will guide my career in the future.

This project has also strengthened my knowledge of experimental management. By streamlining the training process, fixing random seeds and using consistent logging and evaluation, the models were able to be compared fairly. This also allowed them to be reproduced.

The project was not straightforward at times, however. Training all the models across multiple seeds

and different hyperparameters placed a huge burden on the computer I ran it from, which made me trade off in number of episodes and reward thresholds in testing. Also, due to the benchmark being so simple, the DQN and its variants were not allowed to fully demonstrate their performance increases.

In the future, I would like to take the project to a different level through using different benchmarks, such as Atari games, as this field interests me and it would allow me to deeper evaluate each architecture and their strengths. I would also like to experiment with using different loss functions and exploration strategies, which could lead to better learning across the board and would allow me to write a more expansive research paper on this topic.

In conclusion, the project was worked on, and it met its core aims of improving the system and allowed me to produce a comprehensive research paper on the comparison of three different architectural agents. It allowed me to learn more about the topic and influence my career decisions in the future, going into data science.

References

Hu, Y., Huber, A., Anumula, J., and Liu, S-C., 2018. *Overcoming the vanishing gradient problem in the plain recurrent networks*. Zurich: Institute of Neuroinformatics.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M., 2013. *Playing Atari with Deep Reinforcement Learning*. London: DeepMind Technologies.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A., Veness, J., Bellemare, M., Graves, A., Riedmiller, M., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D., 2015. Human-level control through deep reinforcement learning. *Nature* 518, 529-533.

OpenAI, 2025. *Cart Pole*. San Francisco: OpenAI. Available from: https://gymnasium.farama.org/environments/classic_control/cart_pole/ [Accessed 5 March 2025].

Sutton, R. S. and Barto, A. G., 2015. *Reinforcement Learning: An Introduction*. Second Edition. Cambridge, MA: The MIT Press.

Van Hasselt, H., Guez, A., and Silver, D., 2016. Deep Reinforcement Learning with Double Q-Learning. *AAAI-16: Thirtieth AAAI Conference on Artificial Intelligence*, Phoenix, AZ 12-17 February 2016. London: Google DeepMind. Available from: <https://dl.acm.org/doi/10.5555/3016100.3016191> [Accessed 8 April 2025].

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N., 2016. *Dueling Network Architectures for Deep Reinforcement Learning*. London: Google DeepMind.

Watkins, C. J., and Dayan, P., 1992. Q-Learning. *Machine Learning*, 8, 279-292. Boston: Kluwer Academic Publishers.

SeeknnDestroy, 2023. *Deep Q-Network (DQN) for CartPole*. GitHub repository. Available at: <https://github.com/SeeknnDestroy/DQN-CartPole>. [Accessed 3 May 2025].

Appendices

I did not require an ethics checklist for this dissertation.